International Journal of Forecasting 42 (2026) 216–234

Contents lists available at ScienceDirect

International Journal of Forecasting

journal homepage: www.elsevier.com/locate/ijforecast

Hierarchical neural additive models for interpretable demand forecasts☆,☆☆

Leif Feddersen *, Catherine Cleophas

Institute for Business, Kiel University, Olshausenstr. 40, 24098, Kiel, Germany

ARTICLE INFO

Dataset link: https://github.com/leif-feddersen/HNAMs-Demand-Forecasting

Keywords:
Interpretable machine learning
Neural networks
Human–computer interaction
Demand prediction
Retail forecasting
Multivariate forecasting

ABSTRACT

Demand forecasts are the basis for numerous business decisions, ranging from inventory management to strategic facility planning. While machine learning approaches offer accuracy gains, they notoriously lack interpretability and acceptance. To address this dilemma, we introduce hierarchical neural additive models (HNAMs) for time series. HNAMs expand upon neural additive models by introducing a time-series-specific additive model consisting of level and covariate effects. Covariates may interact only according to a user-specified hierarchy. For example, given the hierarchy *weekday, holiday, promotion*, weekday effects are estimated independently, whereas a holiday effect depends on the weekday, and a promotional effect is conditioned on both the weekday and holiday. Thereby, HNAMs clearly attribute additive effects to their respective covariates, enabling intuitive forecasting interfaces with which analysts can interact. We provide benchmarks against established machine learning and statistical models on real-world data to reveal HNAMs' competitive accuracy.

© 2025 The Authors. Published by Elsevier B.V. on behalf of International Institute of Forecasters. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

1. Introduction

Demand forecasting is at the heart of business planning (Mentzer & Moon, 2004). In retail operations, the accuracy and timeliness of the demand forecast affect, for example, the success of supply chain planning and inventory management. Consequently, accurate forecasts enable retailers to consistently offer a wide variety of products to customers without excessive storage or order costs (Fildes et al., 2022b). Thus, accurate demand forecasts are desirable not only for cost control but also for sustaining and increasing customer satisfaction.

However, the proliferation of data sources and the presence of high-dimensional, non-linear relationships significantly increase the complexity of retail environments and, thereby, the challenges for demand forecasting. For example, marketing may affect customer groups and product demand in specific ways depending on the chosen channel, such as newspapers or social media (Neslin & Shankar, 2009). The same goes for holidays and general seasonality, which may change the effect of marketing efforts. Manual and automated approaches to dynamic pricing further increase dimensionality. For example, when the effect of discounts on demand for specific products depends on the seasonality, the pricing of complementary products, and customers' reference prices and thresholds, the relationship between offers and demand becomes not just multi-dimensional but also non-linear (Makridakis et al., 2018a). To handle these challenges, new forecasting approaches are needed.

In that regard, recent results from forecasting competitions point to the superiority of machine learning

☆ The numerical results presented in this manuscript were reproduced by CASCaD on 13 March 2025.

☆☆ We thank Teresa Marquardt, Claudia Ehrig, Ole Grünewald, Sophia Huge, and the anonymous reviewers for their thoughtful feedback on this article.

* Corresponding author.
E-mail addresses: feddersen@bwl.uni-kiel.de (L. Feddersen), cleophas@bwl.uni-kiel.de (C. Cleophas).

https://doi.org/10.1016/j.ijforecast.2025.03.003
0169-2070/© 2025 The Authors. Published by Elsevier B.V. on behalf of International Institute of Forecasters. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

(ML) methods for forecasting, particularly when time series are rich in covariates (Makridakis et al., 2022). Thus, black-box models are outperforming classical statistical time series models such as triple exponential smoothing, ARIMA, and theta, which dominated the field 10 years ago (Makridakis et al., 2018a). Yet, while the increasing availability of high-dimensional data calls for complex ML models, their opacity could ultimately prevent organizations from realizing their potential, due to lack of acceptance.

Due to its relevance to business performance, demand forecasting is frequently subject to managerial oversight and intervention (Fildes et al., 2009). Yet research on judgmental interventions shows that the overall impact on forecast accuracy is questionable (Fildes & Goodwin, 2021), biased, and inefficient (Fildes et al., 2009), and highly dependent on expertise (Edmundson et al., 1988; Sanders & Ritzman, 1992). The call for interpretable, or white-box, forecasts is frequently based on the idea that planners who better understand the forecast are either able to implement better interventions or see less reason to do so at all (Khosrowabadi et al., 2022). However, when black-box ML models outperform interpretable alternatives in terms of accuracy, interpretability comes at a cost.

To address this dilemma for retail demand forecasting, we introduce hierarchical neural additive models (HNAMs). HNAMs combine the quasi-unlimited expressivity of neural networks and the interpretable form of additive models. Thus, they do not aim to be a white-box alternative to ML but rather employ ML to provide insight into the underlying drivers and patterns of demand time series. The resulting forecasts are additively composed of a level term as well as covariate effects that account for interactions across variables over time. To that end, HNAMs let neural subnetworks estimate the covariate coefficients of a linear model where the available information set per subnetwork expands for each covariate according to a user-specified interaction hierarchy.

For example, given the hierarchy *weekday, promotion, holiday*, HNAMs would estimate a coefficient for the weekday effect that only depends on past observations, product attributes, and the weekday itself. The estimated promotion coefficient, one level up in the hierarchy, would crucially account for a weekday-dependent magnitude. Finally, the effect of a holiday depends on its weekday as well as scheduled promotions at the same time. The obtained coefficients are multiplied by the feature values, which are one-hot encodings for categorical values, whereas real values are preferably expressed as relative changes to a baseline (e.g. price deviation from the baseline) to foster sparsity and concise explanations.

By proposing HNAMs as a novel approach to time series forecasting, we estimate individual covariate effects for each forecast and horizon via black-box neural networks accounting for interactions and non-linearities. The gradually expanding information input per covariate network and the fact that coefficients are ultimately multiplied by the input features constrain the models' expressivity and overfitting potential. At the same time, they enable forecast explanations. Thus, the main contributions of this paper lie in

- introducing HNAMs as an interpretable ML approach to retail demand forecasting and documenting their architecture and foundations in existing research,
- demonstrating this functionality on a set of established datasets as well as a dataset provided by an industry partner, and
- benchmarking HNAMs against state-of-the-art ML (specifically, temporal fusion transformers) as well as classical statistical models (specifically, Meta's Prophet, seasonal exponential smoothing, and SARIMAX).

The remainder of this paper is organized as follows: Section 2 motivates the HNAM architecture by discussing related literature on retail demand forecasting and judgment, neural networks in forecasting, and interpretability in ML and forecasting. This section also introduces the existing models that have inspired our proposed architecture. Section 3 details the technical aspects and implementation details of HNAMs. Section 4 introduces the three retail demand datasets and outlines the preprocessing applied. Additionally, Section 5 lists the forecasting and ML models that we benchmark HNAMs against in the computational study. Subsequently, Section 6 presents the results of these benchmarking experiments and illustrates exemplary interpretability outputs. We derive conclusions from these results in Section 7 and discuss limitations, practical challenges, and future research opportunities. The paper's code is available on GitHub.[1]

2. Motivation

To motivate the proposed architecture, we review four interrelated literature streams. Focusing on challenges in retail forecasting, we note that judgmental adjustments are highly prevalent and often necessary when algorithms do not process all available and relevant information. Currently, major developments in ML models, including neural networks, discussed in the second subsection, promise to model such multivariate forecasting problems adeptly. However, their notorious opacity creates another hindrance to successful ML implementations, due to a lack of trust and interaction facilities. The resulting call for more interpretable models is often ill defined, as discussed in Section 2.3. We conclude with initial findings on improved human–computer interaction (HCI) via interpretable models and argue why HNAMs fit the desiderata of interpretable ML.

2.1. Retail demand forecasting and judgment

The literature review by Fildes et al. (2022b) and its postscript (Fildes et al., 2022a) on research and practice in retail demand forecasting provide a comprehensive overview of the field: Demand forecasting supports strategic decisions (location, channel, assortment), tactical decisions (promotions, distribution center inventory planning), and operational decisions (replenishment, workforce scheduling). Commonly, retail demand forecasting

[1] https://github.com/leif-feddersen/HNAMs-Demand-Forecasting

entails demand-altering factors that are known in advance, like promotions, price changes, or holidays. Accounting for these factors in the final forecast requires either the corresponding data and models, or expert adjustments of statistical forecasts. Accordingly, univariate methods like exponential smoothing and ARIMA variants show fair performance at aggregate-level forecasts and for products with low promotional activity, but they fall short for individual products with high promotional activity compared to multivariate counterparts (Ramos et al., 2017).

Recent surveys of software vendors also indicated a growing demand for ML models (Fildes et al., 2020), particularly for interpretable models (Schaer et al., 2022), and stressed the prevalence and importance, but also the risks, of judgmental adjustments. Further industry surveys showed a significant and historically persistent reliance on judgment in the forecasting process (Klassen & Flores, 2001; Sanders & Manrodt, 2003).

A survey by Fildes and Goodwin (2007) of 149 forecasting practitioners found that 75% of forecasting applications involved some form of judgment. Another large-scale evaluation of 60,000 forecasts found that adjustments highly varied in added value, with large, negative adjustments generally improving forecasts (Fildes et al., 2009). Similarly, Petropoulos et al. (2016) investigated how significant errors from judgmental adjustments affect forecasters' subsequent actions: Data from a multinational pharmaceutical company revealed that after experiencing a big loss, experts were inclined to make larger adjustments in the following period, ignoring outcome feedback. This frequently led to further decreases in forecast accuracy. The authors suggest that providing guidance or implementing restrictions within forecasting support systems could mitigate such biases.

While adding extra information to a forecast is a sensible motive, judgmental adjustments are inefficient and often motivated by further, more questionable drivers (Goodwin & Fildes, 2022). These include a general algorithm aversion, i.e., not using or underweighting algorithmic advice after observing inevitable forecast errors (Dietvorst et al., 2015).

In this context, Prahl and Van Swol (2017) found no initial preference for human over algorithmic advice; however, after participants received poor guidance from both, their trust in the algorithm declined significantly more. Trust in automation seems more fragile and adversely affected by inevitable errors than trust in human advisors. We discuss algorithm aversion, its relation to interpretable ML, and potential remedies in Section 2.4. Further adjustment motives pertain to misaligned organizational incentives like gaming the supply chain (Mello, 2009) or signaling one's expertise, as suggested by Fildes and Goodwin (2021).

In a case study, Fildes and Goodwin (2021) examined the demand forecasting process in a pharmaceutical company over 15 years. The company employed a forecasting support system that primarily relied on linear extrapolations. Managers perceived that the system did not capture the non-linear demand patterns associated with product life cycles and market dynamics. The model's inadequacy was a welcome pretext for extensive meetings on forecast adjustments.

Khosrowabadi et al. (2022) analyzed a large retailer's dataset containing adjustments of externally provided multivariate forecasts. Their analysis underscores that large positive adjustments are more frequent but often less accurate, while large negative adjustments are less frequent but more accurate. Further, managers tend to adjust model forecasts given unusual circumstances like extreme weather, promotions, and price changes, despite the model processing these factors—a bias termed 'double (ac)counting' (Goodwin, 2002).

2.2. Neural networks in forecasting

Neural networks have been at the forefront of advancements in domains like natural language processing (NLP) and computer vision (CV); however, they initially faced challenges in the time series forecasting domain. In part due to overconfidence and a lack of rigor among proponents, statistical forecasters met early neural network applications with skepticism (Chatfield, 1993). Systematic evaluations using diverse datasets, such as the M3 competition (Makridakis & Hibon, 2000) and subsequent studies focusing on ML methods, including neural networks (Makridakis et al., 2018a), concluded that ML methods were not yet outperforming their statistical counterparts, despite their theoretical potential.

The M4 competition (Makridakis et al., 2018b) marked a shift towards successful ML applications in forecasting. The winning method combined a recurrent neural network with exponential smoothing (Smyl, 2020), demonstrating the potential of hybrid approaches. Innovations originating from NLP, such as long short-term memory (LSTM) networks (Hochreiter & Schmidhuber, 1997), gated recurrent units (GRUs) (Cho, 2014), and the attention mechanism (Vaswani et al., 2017), have been successfully integrated into time-series-specific neural architectures: Models like DeepAR (Salinas et al., 2020) enable probabilistic forecasting of large datasets, and temporal fusion transformers (TFTs) (Lim et al., 2021) utilize attention to generate forecast explanations.

Neural additive time series models (NATMs) (Jo & Kim, 2023) adapt neural additive models (NAMs) (Agarwal et al., 2021) for time series forecasting by framing the task as a generalized additive mapping from past time steps and covariates to predicted time steps and covariates. In this setup, a shallow neural network processes each past observation for each feature and linearly combines the outputs, resulting in non-linear transformations at the component level. To address the high computational cost that arises from having a separate network for each time step and feature, the original NATM implementation employs weight sharing across time or features. However, it neither explicitly models interactions across features or across time steps, nor does it handle covariates known in the future. By contrast, our proposed HNAM architecture systematically models hierarchical interactions across features and time steps by incorporating sequence-sensible modules, such as GRUs and attention. While preserving the additive spirit underlying NATMs, HNAMs

expand upon it to capture interactions among covariates and across temporal dimensions, enabling more effective multi-step predictions and the integration of future-known covariates. Consequently, HNAMs can address the complex temporal dynamics typical of real-world forecasting scenarios while maintaining the flexibility and interpretability of generalized additive models.

The M5 retail forecasting competition (Makridakis et al., 2022) was dominated by ML methods, predominantly light gradient boosting machines (Ke et al., 2017) and DeepAR. Here, the relative success of ML methods is attributed to the properties of retail datasets. These involve high-dimensional inputs and complex patterns due to the interactions of promotions, price changes, and seasonal effects. ML methods are capable of modeling such non-linear relationships, interactions, and temporal dependencies (Alroomi et al., 2022).

Moreover, the large number of instances, such as unique product and store combinations, allows global ML methods to leverage cross-learning by training on multiple related time series simultaneously, thereby generalizing better than local models (Montero-Manso & Hyndman, 2021). In other words, global ML models offset their enormous parameter count with more training data than low-parameter-count local models for each time series.

Another advantage of neural networks in retail forecasting is their flexibility with respect to the types of inputs and outputs they model. For example, using a quantile loss function allows neural networks not only to predict average demand but also to prescribe cost-sensitive order quantities in data-driven newsvendor settings (Feddersen & Cleophas, 2023; Qi et al., 2023; Ulrich et al., 2021).

Despite these advantages, the lack of interpretability in ML methods is an often-cited challenge in practical implementations, where understanding the rationale behind forecasts is crucial (Fildes et al., 2022a). Their black-box nature can hinder trust and acceptance among practitioners, leading to resistance to adopting these advanced methods.

2.3. Interpretability in ML and forecasting

Most researchers and practitioners agree that ML methods often lack interpretability. However, the concepts of interpretability, explainability, and transparency are frequently used without clear definitions or distinctions. Following Rudin (2019), we refer to interpretable models as those that produce explanations to help a human understand the model's output. Models are furthermore transparent when their inner workings are fully or partially traceable—termed 'simulatability' and 'decomposability', respectively, by Lipton (2018). Murdoch et al. (2019) define interpretable ML as 'the extraction of relevant knowledge from a machine-learning model concerning relationships either contained in the data or learned by the model'. They outline three key desiderata for interpretability:

- Predictive accuracy: The model should accurately predict its targets, as measured by appropriate error metrics.
- Descriptive accuracy: The interpretation should truthfully describe the model's decision-making process, accurately reflecting how it arrives at its predictions.
- Relevancy: The interpretation should provide useful insights to its intended audience, aiding their understanding and decision making.

The literature on interpretable ML suggests two primary tradeoffs when considering interpretable models and methods. First, there is tension between predictive accuracy and descriptive accuracy. Simpler models allow for more truthful descriptions of their inner workings, thus increasing descriptive accuracy, but may achieve lower predictive accuracy than more complex models. Murdoch et al. (2019) emphasize that accurate descriptions of inaccurate predictions can never be relevant. Second, there is tension between the understandability of an explanation and its completeness (Gilpin et al., 2018). A highly understandable explanation may oversimplify the model, lacking completeness in describing its behavior. Conversely, an interpretation can be complete—fully capturing all nuances of the model—but may become too complex for practical comprehension.

Existing approaches to interpretability in time series forecasting include proxy methods like LIME (Ribeiro et al., 2016) and SHAP (Lundberg & Lee, 2017), which provide post hoc explanations. Antipov and Pokryshevskaya (2020) used SHAP values to generate explanations for retail forecasts, emphasizing the impact of promotional effects. However, as a post hoc method, SHAP only approximates the actual model's behavior and may not capture feature interactions.

Rudin (2019) argued for the use of inherently interpretable models over post hoc explanations for black-box models. The author contended that post hoc explanations may result in misleading interpretations, particularly in high-stakes decision-making contexts where trust and transparency are essential. In contrast, inherently interpretable models provide transparency directly through their structure and functioning, ensuring that interpretations are accurate reflections of the model's operations. Murdoch et al. (2019) defined the following key methods for model-based interpretability:

- Sparsity: Limiting the number of parameters or features to simplify the model and make it easier to interpret.
- Simulatability: Designing models simple enough for humans to simulate and comprehend their entire decision-making process.
- Modularity: Building models with components that can be interpreted independently, allowing for partial understanding of complex models.
- Domain-based feature engineering: Leveraging domain knowledge to create features that enhance the interpretability and relevance of the model.

Simulatability is rarely encountered in time series forecasting, even with classical statistical models. Arguably, single exponential smoothing or rolling average calculations can be traced entirely by the average stakeholder.

Below, we discuss how the remaining key methods relate to forecasting approaches. Meta's Prophet can be considered a sparse and modular model which uses domain-based feature engineering in the form of Fourier transformations to model seasonality (Taylor & Letham, 2018). Its ease of use and intuitive model structure make it a popular choice for applied forecasting. At the same time, the simple structure can miss important non-linear effects and interactions. Further, Prophet has to be individually fitted for each time series, foregoing the benefits of cross-learning. These drawbacks also apply to traditional models like ARIMAX and SARIMAX (Box et al., 2015), which are particularly time consuming to fit with parameter optimization. While vector autoregression models (Lütkepohl, 2005) handle multiple time series and their interactions simultaneously, they still lack the ability to model non-linear relationships. Non-linearity is handled by several shallow neural networks in NATMs (Jo & Kim, 2023), which attribute their outputs to the respective input features and time steps. Thus, they maintain modularity but give up on sparsity.

A weaker form of explanations can be derived from transformer models like the TFT (Lim et al., 2021). Its attention weights provide some interpretability by indicating which time steps the model 'attends' to for a given forecast; however, the exact contributions of each feature and time step remain unclear due to prior and subsequent non-linear processing (Feddersen & Cleophas, 2023). ML approaches such as gradient boosting machines (e.g. LightGBM) (Ke et al., 2017) can model non-linear effects and interactions between variables but do not provide interpretability insights beyond feature importances.

2.4. Improving HCI with interpretable models

The literature discussed thus far suggests that the HCI in retail forecasting frequently entails judgmental adjustments motivated by an actual, perceived, or suggested inadequacy of the automated model. This is supported by extensive research on algorithm aversion, summarized in Burton et al. (2020), which proposes that the apparent mismatch between the mental model of the human decision maker and how they perceive the algorithmic model is a main driver for the aversion. A typical proposed scheme is that initially inflated expectations regarding the model's capability are necessarily disappointed. Subsequently, the lack of cognitive compatibility between humans and algorithms prevents further inquiry or reconciliation.

Interpretable models, insofar as they approximate a decision maker's mental model, promise to alleviate algorithm aversion. For the present retail forecasting context, we and the managers we spoke to tend to think of the demand forecasting task as analyzing individual demand drivers (known covariates) and their interactions, and adding them to the baseline sales. This is also evident in the data by Khosrowabadi et al. (2022) in which managers attempted to include information that was already incorporated by the model. The authors suggest that an interpretable model could alleviate this bias by clearly showing the covariates' contributions. Interpretable models would further enable more structured and targeted adjustments, aligning with Dietvorst et al. (2018)'s finding that algorithm aversion is reduced if users can adjust outputs.

However, interpretable models do not increase acceptance under all circumstances: Lehmann et al. (2022) experimentally demonstrated that a model that seems insufficiently complex for the task at hand, regardless of its actual adequacy, will be less utilized once explained. Unfortunately, behavioral research on improved acceptance and forecasting outcomes with interpretable models is scarce.

A related case study by Vössing et al. (2022) found that a more transparent forecasting interface, which lists the considered covariates for a forecast, results in more favorable adjustments. The authors concluded that their transparent user interface allows managers to calibrate their interventions more precisely and productively. That is, they become more attuned to the circumstances under which their extra-model information (or intuition) can, in fact, improve the forecast while abstaining from adjustments otherwise.

Examining the impact of structured adjustment facilities, Marmier and Cheikhrouhou (2010) developed a structured approach to guide forecasters in making adjustments based on expert knowledge of event effects. They identified four types of factors—transient, transferred impact, quantum jump, and trend change—that can lead to significant and predictable changes in forecasts not captured by purely statistical methods. Their case study demonstrated that structured integration of managerial judgment significantly improves forecast accuracy compared to unaided adjustments and purely statistical forecasts.

Similarly, Trapero et al. (2013) investigated the effectiveness of judgmental adjustments to statistical forecasts in the context of promotional events. They found that while managerial adjustments can sometimes improve forecast accuracy during promotions, this improvement is not consistent. Specifically, overly large adjustments tend to reduce forecasting accuracy. The authors demonstrated that multivariate statistical models, such as transfer function models incorporating past promotion information, achieve lower overall forecasting errors than both unaided statistical forecasts and judgmentally adjusted forecasts. They suggested that structured statistical approaches can effectively replace managerial adjustments in forecasting promotional sales, leading to improved forecast accuracy and reducing the workload on managers.

Examining not judgmental adjustments but direct judgmental forecasting, both Lawrence et al. (1985) and Carbone and Gorr (1985) compared judgmental and statistical forecasts with M1 competition data (Makridakis et al., 1982). They found contrary results explained by the respective forecasting interfaces: Unlike the unaided interface of Carbone and Gorr (1985), the study by Lawrence et al. (1985) let participants sequentially extrapolate the components of the time series, including trend and seasonality, through a tabular or graphical presentation, thus facilitating comparisons and the identification of seasonal patterns.

Finally, a behavioral study on transparent forecasting interfaces (Feddersen, 2024) demonstrated marginally better and less variable forecasts after adjustments with a transparent interface. Respondents remarked upon their appreciation for forecast explanations, but some felt overwhelmed, highlighting that explanations must be accompanied by adequate training.

2.5. Relevance of the proposed architecture

In conclusion, existing research highlights a dual goal for forecasting algorithms: achieving high prediction accuracy while maintaining interpretability. Lack of acceptance and consequent unnecessary adjustments result from interpretable but too simplistic models as well as complex but too opaque models. Post hoc explanations fall short in descriptive accuracy, leading scholars to call for inherently interpretable models. Addressing these outlined challenges and the interpretability desiderata from Murdoch et al. (2019), we introduce hierarchical neural additive models (HNAMs), which combine the predictive power of neural networks with inherent interpretability. HNAMs process known covariates through individual attention-based subnetworks, ensuring that only active covariates contribute non-zero effects. Interactions are hierarchically structured to retain interpretability, and all covariate effects are additively combined with a baseline level derived via smoothing. Thus, by modeling nonlinearities, temporal dependencies, and interactions, the architecture promises high predictive accuracy.

It also ensures descriptive accuracy through interpretability inherently arising from the model's additive structure. The clearly attributable covariate effects allow practitioners to partially trace the model's decision-making process through modularity. The additive composition aligns with managers' reasoning, making the interpretations relevant and easily understandable to the intended audience (relevancy). By matching the algorithmic structure with the manager's mental model, HNAMs promise improved cognitive compatibility and perceived control (Burton et al., 2020) to reduce algorithm aversion and avoid adjustment biases like double counting (Khosrowabadi et al., 2022).

By filling the gap between opaque ML and interpretable statistical models, HNAMs provide a balanced approach that meets the dual goal of accuracy and interpretability identified in the literature. The next section details its technical implementation.

3. Proposed architecture

In describing the proposed HNAMs architecture, we first present a conceptual overview and differentiate covariate types as well as the functional components processing them. Afterward, we describe our proposed implementation of the functional components in detail.

3.1. Covariates and components

Conceptual overview. Fig. 1 provides a high-level illustration of the HNAM architecture. The model distinguishes covariates based on their temporal availability and whether they should have an individual effect, termed causal covariates.

To model interactions among causal covariates in a structured and interpretable manner, HNAMs require these covariates to be specified in a user-defined interaction hierarchy. In this hierarchy, each causal covariate's estimated effect can depend on itself and on causal covariates lower in the hierarchy, but not on those higher up. This ensures that interactions are systematically incorporated while maintaining interpretability.

We recommend formulating the hierarchy by ordering covariates in descending frequency of their occurrence. For example, consider the hierarchy: *weekday* (most frequent), *promotion*, *holiday* (least frequent). In this setup, the effect of the weekday covariate is estimated consistently independent of other causal covariates, resulting in more stable explanations. The effect of a promotion is conditioned on the weekday, capturing how promotions might have different impacts depending on the day of the week. The effect of a holiday is conditioned on both the weekday and promotion, allowing the model to capture complex interactions such as holiday promotions on specific weekdays.

For each causal covariate, HNAMs employ an attention-based coefficient network. Each network processes the causal covariate itself, those lower in the interaction hierarchy, and past, static, and non-causal covariates, distinguished below. Their outputs are coefficients per time step and covariate, which are multiplied with the respective values, detailed below, to obtain the contributed effect per covariate.

The level component of HNAMs forecasts the baseline demand by dynamically applying a variant of double exponential smoothing with trend damping to the cleaned (i.e., corrected for known covariate effects) sales history. The layer adaptively computes smoothing parameters from the observed history using a gated recurrent unit (GRU). The projected level serves as a baseline to capture the general demand level and trend changes. Sudden level changes may also result from unexplained factors and can prompt a re-evaluation of covariates. Furthermore, the exponential smoothing operation is familiar to forecasting professionals and straightforward to interpret.

Summing up the outputs of level and covariate networks yields the final forecast. Below, we distinguish the covariates and explain their processing in detail.

Causal covariates. Causal covariates are features known in both the past and future that are expected to affect demand. Examples include date-derived features (weekday, season, holiday indicators) and planned promotional activities (price changes, advertisements). Note that our model does not prove or assume causality in the strict sense, since it receives assumed influential covariates as a hyperparameter. However, it clearly differentiates and quantifies the model-learned and predicted contributions per covariate. Matrix C organizes causal covariates per the

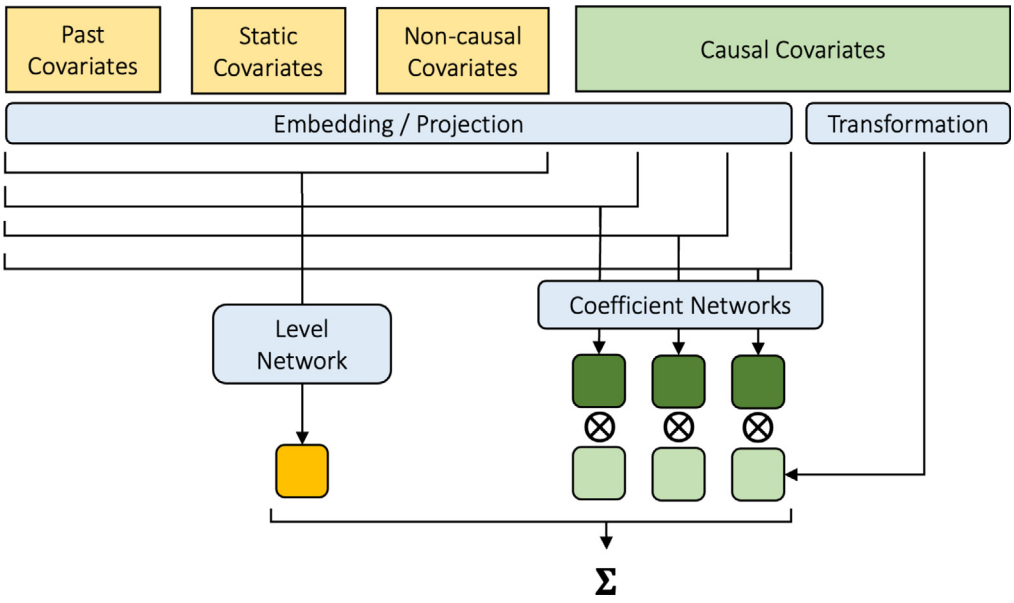


Fig. 1. Conceptual overview of the HNAM architecture.

interaction hierarchy across n_C rows, and spans $T_h + T_f$ columns for the time steps, where T_h is the historical period and T_f is the forecast horizon.

Notation. For ease of notation and since batches do not interact, we leave out the leading batch dimension B here and in the following. Further, we adopt a Python-inspired slicing notation for subsetting rows or columns in our matrices. For instance, $C[:i+1]$ refers to the submatrix of C containing rows from index 0 up to and including index i (a total of $i+1$ rows), since the upper bound $i+1$ is excluded in Pythonic slicing. Similarly, $X[:, :T_h]$ refers to all rows of X but only the first T_h columns (i.e., time steps from index 0 to $T_h - 1$). The operator $\odot$ denotes the element-wise (Hadamard) product between two arrays of the same shape. For arrays A and B, it is defined as $(A \odot B)_n = A_n \cdot B_n$ for all indices n. Additionally, we use standard mathematical functions such as the sigmoid function $\sigma(x) = \frac{1}{1+e^{-x}}$, the softmax function $\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$, and the cumulative sum function $\text{cumsum}(x)$, where $\text{cumsum}(x)_t = \sum_{k=0}^{t} x_k$.

Non-causal covariates. Non-causal covariates are features known in both the past and future that do not require individually estimated effects within the model. They provide essential temporal context, especially in non-recurrent architectures that lack inherent temporal ordering mechanisms (Vaswani et al., 2017). Examples include simple time indices (e.g. integers representing time steps) or continuous representations of time (e.g. sinusoidal yearly patterns). While non-causal covariates do not have individually estimated coefficients, they influence the model by modulating the estimated effects of causal covariates. By providing temporal information, non-causal covariates enable the model to adjust the impact of causal covariates based on underlying temporal patterns and order. For instance, the effect of a promotion (a causal covariate) on demand may change over time, due to trends or seasonal effects not explicitly captured by causal covariates. Non-causal covariates supply the necessary temporal context that allows the model to capture these variations, effectively shaping how the causal effects are estimated. Matrix T stores these non-causal covariates across $T_h + T_f$ columns.

Static covariates. Similarly, matrix S stores time-invariant covariates, typically product and store attributes. These covariates are repeated across $T_h + T_f$ columns to align with the temporal dimensions of other inputs.

Past covariates. Finally, matrix P contains covariates only known in the past. It typically contains the past demand observations alongside other covariates that are not known in advance. Similar to non-causal and static covariates, past covariates have no individually estimated effects but modulate causal covariate effects. For more convenient indexing below, we let P have $T_h + T_f$ columns where values in the future are zero and will not be used for forecasts.

3.2. Implementation

Functional components. Covariates are first scaled to the model's embedding size via projection or embedding. In order to produce a forecast vector y of T_f forecast steps, the respective tensors, representing information subsets, pass through *coefficient* neural networks f_i with one neural network for each of the n_C causal covariates and a *level* neural network g, the results of which are finally summed.

In the following, we detail the implementation of these functional components.

Embeddings and projections. The input layer projects all covariates into the model's embedding size E, a hyperparameter that determines the model's internal dimensionality and, thereby, expressiveness. To achieve this, an embedding table handles categorical covariates. Embeddings are a denser alternative to one-hot encodings to translate categorical values into a numerical representation. Continuous covariates are projected into the model's embedding size through an affine transformation.

Assembling information tensors. The constant embedding size of embedded covariates allows them to be summed in the feature dimension. Whenever HNAMs require a certain information set, the respective covariates are summed and passed through a layer normalization. We omit that layer normalization in the notation below for brevity.

MLP. At various points in our architecture, we employ a multi-layer perceptron (MLP) for non-linear processing. Specifically, the input, originally in the model's embedding size, is projected into a higher-dimensional space by increasing its size by a user-defined expansion factor—a hyperparameter that controls the degree of dimensionality expansion within the MLP layers. This practice is standard in transformer architectures and increases non-linear processing capabilities (Vaswani et al., 2017). For instance, an expansion factor of four quadruples the embedding size in the MLP layers compared to the input embedding size. The expanded representation then passes through a Gaussian error linear unit (GELU) activation function to introduce non-linearity. Afterward, it is projected back to the original embedding size with dropout applied. This dimensionality expansion allows the MLP to process information in a more complex and non-linear manner, contrasting with the attention layers that primarily facilitate information exchange across different temporal positions in the input.

GRU. The gated recurrent unit (GRU), introduced by Cho (2014), is a recurrent layer to efficiently capture dependencies in sequential data. GRUs address the vanishing gradient problem by employing gating mechanisms that regulate information flow. They simplify the LSTM architecture introduced by Hochreiter and Schmidhuber (1997) by using just two gates—an update gate and a reset gate—and merge the cell and hidden states. This reduces computational complexity while achieving performance comparable to that of LSTMs.

Temporal convolution layer. This layer is used in the attention block described below to extract local features across adjacent time steps. It expands its input using a one-dimensional convolution with a kernel size of three, accounting for the last two time steps with zero padding for the two oldest observations. Post-convolution, a GELU activation and dropout regularize the output before it is projected back to the embedding size.

Attention. The coefficient networks incorporate information from other time steps globally via multi-head attention (Vaswani et al., 2017) with a temporal convolution to calculate queries, keys, and values. Our attention implementation is standard but ensures efficiency by letting the individual covariate networks share the weights for the projection of queries, keys, and values and computing attention scores in parallel across all covariate effects. Note that this does not imply any information exchange between the subnetworks.

Coefficient networks. Each coefficient neural network f_i observes S, T, and P completely, in addition to the ith causal covariate as well as causal covariates below in the interaction hierarchy.

Queries, keys, and values for the ith covariate and coefficient network are obtained as:

$$
\begin{aligned}
Q &= \text{TemporalConv}\left(S + T + C[:i+1]\right), \\
K &= \text{TemporalConv}\left(\left(S + T + P + C[:i+1]\right)[:, :T_h]\right), \\
V &= \text{TemporalConv}\left(\left(S + T + P + C[:i+1]\right)[:, :T_h]\right).
\end{aligned}
$$

Conceptually speaking, queries are the information we have about the forecast horizon. Keys and values additionally contain historically observed information. The attention operation then extracts relevant (i.e., similar query and key pairs) information (represented in the values) and incorporates it into the queries. Multi-head attention facilitates multiple such operations in parallel. For that, the sequence is split in the channel dimension into multiple heads, and scaled dot-product attention is computed for each head:

$$
\begin{aligned}
Y &= \text{attention}(Q, K, V) \\
&= \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V,
\end{aligned}
$$

where d_k is the dimension of each head.

The output O is then reassembled by stacking the heads in the channel dimension. We apply a residual connection by adding the projected attention output back to the queries:

$$
\begin{aligned}
O_{\text{proj}} &= \text{Dropout}\left(\text{Proj}(Y)\right), \\
o &= Q + O_{\text{proj}}.
\end{aligned}
$$

Next, we process o through a layer normalization and MLP, followed by another residual connection to obtain the latent covariate effects LatCoeff:

$$
\text{LatCoeff} = o + \text{MLP}\left(\text{LayerNorm}(o)\right).
$$

For each covariate i in $\{0, 1, \ldots, n_C - 1\}$, we obtain its coefficient by selecting its latent coefficient LatCoeff_i from LatCoeff, passing it through an individual MLP, and projecting it to the required dimensionality:

$$
\text{Coeff}_i = \text{Proj}_i\left(\text{MLP}_i\left(\text{LatCoeff}_i\right)\right).
$$

Here, MLP_i is the multi-layer perceptron specific to covariate i, and Proj_i accounts for the covariate's cardinality and the required output size (e.g. for quantile forecasting).

Coefficients are then multiplied with the covariate's transformed value $t(C[i])$. For categorical covariates, these transformed values are $k - 1$ one-hot encodings so that they are zero for baseline scenarios (e.g. no promotion, no holiday). Real values are expressed as relative changes (e.g. the percentage change in price relative to the last 14 days, also ensuring zero for baseline scenarios):

$$
\text{Effect}_i = \text{Coeff}_i \odot t(C[i]).
$$

The multiplication ensures sparse outputs such that non-zero effects only occur for active covariates. For instance, if a causal covariate represents a binary feature like a promotion or holiday indicator, multiplying the coefficient with the covariate ensures that the effect is present only when the covariate is supposed to have an effect (e.g. when the indicator is 1) and nullified when it is inactive (e.g. when the indicator is 0). This mechanism maintains the interpretability of the model by ensuring that the estimated effects correspond to their respective covariates.

Level network. The level network component g generates the forecasted demand baselines by dynamically applying a variant of double exponential smoothing, where the smoothing parameters are adaptively learned from covariate embeddings through a GRU network. This approach allows the model to capture both the level and trend components of the time series, informed by relevant covariate information.

The input to the level component's smoothing calculations includes only information from the encoder sequence—the known past—so that it does not implicitly model the causal covariates for which we want to estimate individual effects. Smoothing is applied to the historical sales y_{past}, cleaned by estimated covariate effects:

$$y_{\text{cleaned}} = y_{\text{past}} - \sum_{i=0}^{n_C-1} \text{Effect}_i[:, : T_h].$$

Smoothing parameter generation: We compute the weights for the level component by passing only past available information to a GRU layer, which produces the final hidden state h:

$$h = \text{GRU}\left((S + T + P + C)\,[:, : T_h]\right).$$

This hidden state passes through a linear layer followed by a sigmoid activation to produce the smoothing parameters α, β, and the damping parameter ϕ (if using a damped trend):

$$[\alpha, \beta, \phi] = \sigma\left(\text{Linear}(h)\right).$$

Level calculation: We compute the weights for the level component using α over the encoding range $k = 0, \ldots, T_h - 1$ and normalize them:

$$w_k^{(L)} = \alpha\,(1-\alpha)^{T_h-1-k},$$

$$w^{(L)} = \frac{w^{(L)}}{\sum_{k=0}^{T_h-1} w_k^{(L)}}.$$

The projected level l_{level^*} is computed by applying these weights to the past level values y_{cleaned}:

$$l_{\text{level}^*} = \sum_{k=0}^{T_h-1} w_k^{(L)} \cdot y_{\text{cleaned}}[k].$$

This scalar value is then repeated across the forecast horizon of length T_f to form the level component l_{level} of the forecast sequence.

Trend calculation: Differences between consecutive past levels are calculated to obtain the trend components:

$$\Delta y_{\text{cleaned},k} = y_{\text{cleaned},k} - y_{\text{cleaned},k-1}, \quad k = 1, \ldots, T_h - 1.$$

Weights for the trend component are computed using β and normalized:

$$\tilde{w}_k^{(T)} = \beta\,(1-\beta)^{T_h-1-k}, \quad \text{for } k = 1, 2, \ldots, T_h - 1,$$

$$w_k^{(T)} = \frac{\tilde{w}_k^{(T)}}{\sum_{j=1}^{T_h-1} \tilde{w}_j^{(T)}}.$$

The weighted sum of trends is computed:

$$l_{\text{trend}^*} = \sum_{k=1}^{T_h-1} w_k^{(T)} \cdot \Delta y_{\text{cleaned},k}.$$

This trend component is then extended over the forecast horizon. If using a damped trend, it is adjusted using ϕ. The cumulative sum gives the trend over the forecast horizon:

$$l_{\text{trend}} = \text{cumsum}\left(l_{\text{trend}^*} \cdot \phi^t\right),$$

where $t = [0, 1, \ldots, T_f - 1]$.

Level forecast: The final forecasted level over the horizon is obtained by combining the projected level and trend components:

$$y_{\text{level}} = l_{\text{level}} + l_{\text{trend}}.$$

This results in a forecast sequence that accounts for both the estimated level and the trend over the prediction horizon. By parameterizing the smoothing factors α, β, and ϕ based on covariate information through the GRU, the model adapts to varying patterns in the data, flexibly modeling level and trend components, effectively merging the interpretability of exponential smoothing with the adaptability of neural networks.

Forecast. HNAMs derive a forecast vector y as a component-wise calculation and subsequent addition of individual covariate effects and the projected level:

$$y_{\text{effects}} = \sum_{i=0}^{n_C-1} f_i\,(S, T, P, C[: i+1]) \odot t(C[i]),$$

$$y_{\text{level}} = g\left((S + T + P + C)\,[:, : T_h], y_{\text{effects}}[:, : T_h]\right),$$

$$y = y_{\text{level}} + y_{\text{effects}}[:, T_h :].$$

4. Datasets and preprocessing

For robust benchmarking, we generate and evaluate forecasts in three different retail demand datasets. All datasets have multiple covariates alongside the recorded sales data. Two datasets are publicly available and one stems from an industry partner.

In the following, we provide an overview of the preprocessing we apply in this study, and we highlight how time series are filtered, what covariate features are used, and how a rolling-forward evaluation is carried out.

4.1. Datasets

Walmart. This public dataset (University of Nicosia, 2020) consists of hierarchical sales data provided by Walmart through the University of Nicosia as part of the M5 forecasting competition. It spans 1941 days from 2011 to 2016. The dataset includes daily item-level sales data for 3049 products and 10 store locations across three states.

Favorita. The Favorita dataset (Corporación Favorita, 2018) is publicly available and provided by Corporación Favorita, an Ecuador-based grocery retailer, for a forecasting competition. It spans 1684 days from 2013 to 2017 across 4035 products and 54 stores.

Retail. We further evaluate models on a dataset provided by a medium-sized retailer. It spans 2472 days from 2015 to 2022, covering 2910 products across three stores.

4.2. Data selection

The research documented in this paper focuses on the interpretability aspect of forecasting to enhance human–computer interaction. To create a fitting benchmarking environment for this purpose, we select time series (i.e., unique product and store combinations) that are likely to be adjusted due to their economic relevance and for which we can assess the plausibility of interpretability outputs.

We exclude time series with apparently low or intermittent sales. Since all datasets lack pertinent information as to whether a product was out of stock or not offered at all, it becomes challenging to create realistic benchmarks for forecasting consumer demand. Since we cannot differentiate these scenarios without making substantial assumptions, we focus on products with more stable sales, assuming they were less frequently out of stock.

Further, we argue that daily forecasts of intermittently demanded products are generally less relevant than non-intermittent ones. For instance, products with high daily demand and a short shelf life, such as fruits, vegetables, and dairy products, typically do not exhibit intermittency. On the other hand, products with slower turnover rates, often due to higher costs or specificity, usually have longer shelf lives, like spices, beverages, and non-food items. For these products, aggregate forecasts are more relevant, making the problem of intermittency conveniently disappear.

Our data selection process limits selected time series by the criteria above while retaining at least 500k samples, similar to the experimental setup by Lim et al. (2021). Concretely, our filtering sequence involves the following:

1. We ignore all sales that occur before any two-week period without sales.
2. A time series must have enough observations to be considered during hyperparameter optimization.
3. We set a minimum median sales threshold for inclusion, starting at five and increasing it by one in each step. We continue this process until the number of available samples drops below 1500k.
4. We increase a threshold for acceptable missing values in small increments beginning at 0% until at least 500k samples are obtained.
5. Missing sales are subsequently filled with the rolling 14-day average of sales.

This selection process yields 417 (Walmart), 665 (Favorita), and 350 (Retail) individual time series across 526k, 712k, and 539k samples, respectively.

Table 1
Covariates entering multivariate models per dataset.

	Walmart	Favorita	Retail
Categorical			
Product	✓	✓	✓
Store	✓	✓	✓
Promotion	✗	✓	✓
Weekday	✓	✓	✓
Holiday[a]	✓	✓	✓
SNAP[b]	✓	✗	✗
Continuous			
Sales (only past)	✓	✓	✓
Relative price[c]	✓	✗	✓
Oil Price (only past)	✗	✓	✗

[a] For Favorita, we differentiate local, regional, and national holidays as separate covariates; for Walmart, we differentiate sporting, cultural, and national events or holidays. We add features for the five days prior to each holiday.
[b] The Supplemental Nutrition Assistance Program (SNAP), previously known as the Food Stamp Program, provides funds to low-income households specifically for purchasing food. The feature marks the days on which funds are distributed.
[c] The percentage deviation of the day's price from the rolling 30-day mean.

4.3. Features

While most covariates utilized by the benchmarked models are available in all datasets, some are dataset-specific. Unless indicated otherwise, all covariates are available not only for past observations but also in the prediction horizon. Table 1 lists the covariates, differentiates their type as categorical or continuous, and indicates their availability per dataset. In addition to the listed covariates, neural networks use absolute and relative time indices as well as sine and cosine embeddings of the day of the year to provide temporal context to the models.

4.4. Evaluation

Across all datasets, we use the last five fully available months as test sets. For each test month, we train the neural networks using observations up to two weeks before the start of the test month and use the two weeks immediately preceding the test month as a validation set for early stopping. As detailed in Appendix, we perform hyperparameter optimization for the neural networks using the last two months of the training set.

We generate daily forecasts with a maximum horizon of two weeks via a sliding window in each test month without retraining. We fine-tune with updated data prior to each new test month. Similarly, we keep the hyperparameters for the remaining benchmark models constant per test month but use all data up to the respective forecast date for training.

We aim to forecast sales with daily granularity. While coarser temporal aggregations can be sensible depending on the business case, we focus on our model's interpretable forecasts based on covariates of daily frequency. Overall, our evaluation methodology is designed to closely replicate a realistic forecasting environment in the retail sector, where models are periodically updated but must

generate frequent and accurate forecasts without constant retraining.

Although HNAMs support quantile prediction, we limit our exploration to forecasting expected sales by providing point forecasts. This choice allows for clearer error metrics and interpretability outputs in this paper.

5. Benchmarked models

The following section presents an overview of dataset-specific HNAM implementation details and introduces the considered benchmark models. These include TFT, Prophet, ETSX, lasso, and SARIMAX.

Hierarchical neural additive models. As outlined in Section 3, HNAMs distinguish different types of covariates and structure their interactions through an interaction hierarchy. We use the same hierarchy for all datasets, provided that the respective covariate is available: *weekday, promotion, relative price, holiday*. This represents an ordering by decreasing frequency of occurrence, as argued for in Section 3.1.

The Walmart dataset does not include information on scheduled promotions. However, SNAP indicates days when eligible customers received financial support for groceries. Holidays are differentiated into three separate covariates in Walmart and Favorita, distinguishing them by type and region, respectively.

Temporal fusion transformer. The temporal fusion transformer (TFT) (Lim et al., 2021) is a deep learning architecture that employs the transformer architecture for time series forecasting. Its introductory paper demonstrated TFT outperforming other popular neural networks for forecasting, including DeepAR (Salinas et al., 2020). TFT, like HNAMs, differentiates covariates with respect to their availability during the forecast horizon. Notably, its attention mechanism and variable selection networks offer some interpretability; however, the explanations are significantly less immediate than in our proposed architecture (Feddersen & Cleophas, 2023).

Neural network implementations. The two neural network-based methods above are implemented in PyTorch Forecasting (Beitner, 2020). We optimize hyperparameters on the last two months of the training data via a random search with a hyperparameter grid setup similar to that of Lim et al. (2021). Each search runs for 24 h. The respective grids are provided in Appendix.

For the first test month, we train up to 300 epochs, applying early stopping based on the loss in the validation set (i.e., the last two weeks of each training set) with a patience of 30 epochs. We select the best model with respect to the validation loss. For the remaining test months, neural networks are fine-tuned for up to 100 epochs, employing the same early stopping logic. Models are trained on a high-performance cluster node with an NVIDIA H100 GPU.

The statistical methods introduced below are each trained with parallelization on the time series level on an Intel Cascade Lake node.

Prophet. Prophet (Taylor & Letham, 2018) decomposes a time series into trend, multiple seasonalities, and holiday components. The model conveniently handles missing values and is intended to allow fast human-in-the-loop iterations. We use Prophet's Python implementation (Taylor & Letham, 2017) with standard parameters. We validate an additive versus a multiplicative model in the training data for each time series and use the better-performing model to generate forecasts.

ETSX. Seasonal exponential smoothing (also termed ETS, for error, trend, and seasonality) applies exponentially decaying weights to the error, trend, and seasonal components of univariate time series. We use the statsforecast (Garza et al., 2022) AutoETS implementation to automatically determine the additive or multiplicative configuration of error, trend, and seasonality via the Akaike information criterion. In addition to univariate ETS models, we fit a linear regression on the residuals to account for known external covariates, and refer to these models as ETSX (ETS with eXogenous variables).

SARIMAX. Seasonal autoregressive integrated moving average with exogenous regressors (SARIMAX) models represent the detrended time series as a combination of autoregressive and moving average processes alongside a seasonal component and external regressors. The respective parameters are found with the statsforecast AutoARIMA implementation.

Lasso regression. We also test a lasso regression with the known covariates and time series features (sales lags of 1, 3, 7, and 14 days, and rolling averages of 7 and 14 days). The model autoregressively forecasts the horizon. The time series features are recalculated accordingly at each step in the horizon using the forecasted values. A five-fold temporal cross-validation determines the regularization parameter.

6. Results

This section presents the results of our computational study. We compare computational times and error metrics for the benchmarked models. Further, we discuss HNAM interpretability outputs at different aggregation levels.

6.1. Computational times

Table 2 summarizes the training durations per model and dataset. As expected, the statistical models—lasso, ETSX, Prophet, and SARIMAX—train substantially faster than the neural network models, HNAMs and TFT. Among the statistical models, lasso and ETSX are the quickest and considerably faster than SARIMAX, particularly in Favorita's most extensive dataset.

The neural networks exhibit much longer training times. HNAMs' total training time is roughly 5 to 10 times longer than SARIMAX's, while TFT takes even longer, with training times approximately five times longer than HNAMs'. This ratio persists on the per-epoch training time. Despite the neural networks' relatively longer training times, one should note that pre-trained models may be fine-tuned to new data with lower computational costs. Also, inference takes only a fraction of the training time.

Table 2
Training times in seconds.

		Favorita	Walmart	Retail
Training time	SARIMAX	8,660	6,559	2,529
	ETSX	491	241	228
	Lasso	234	140	87
	Prophet	3,029	1,474	961
	HNAMs	17,913	15,751	12,154
	TFT	108,828	55,964	68,558
per epoch	HNAMs	68	78	52
	TFT	470	328	295

6.2. Aggregate error metrics

This section presents aggregate error metrics of the considered models on the test sets per dataset. Negative predictions are truncated at zero. Alongside the optimized error metric root mean squared error (RMSE), standardized per time series for comparability (sRMSE), we also present the root mean squared scaled error (RMSSE) used in the M5 competition, which represents forecast performance relative to a one-day naive forecast.

For a single time series, the RMSE between the predicted values $\hat{y}_i$ and the actual values y_i for $i = 1, \ldots, n$ penalizes large deviations more and serves as a loss function in the neural networks' training process. It is defined as follows:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2}$$

RMSE scores are calculated on z-score-normalized forecasts and actuals, mirroring the training setup for the neural networks, reported as the sRMSE.

Similarly, the RMSSE (root mean squared scaled error) normalizes errors by the mean absolute error of a naive one-step in-sample forecast, enabling comparison across time series with different scales. It is defined for a single time series as follows:

$$\text{RMSSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(\frac{\hat{y}_i - y_i}{s}\right)^2}$$

where the scaling factor s is calculated as the mean absolute difference of the naive one-step forecast:

$$s = \frac{1}{n-1}\sum_{i=2}^{n}|y_i - y_{i-1}|$$

For all error metrics, we report the mean $\bar{x}$ and median $\tilde{x}$ across all time series. To convey another robust assessment of model performance, we report the frequency of first and second ranks achieved across time series when ranking models by RMSE per time series.

The aggregate error metrics presented in Tables 3, 4, and 5 indicate that the advanced neural network models, TFT and HNAMs, consistently outperform the linear benchmarks across all datasets. TFT generally achieves the best performance, notably in the Favorita and Retail datasets, where it records the lowest mean and median RMSSE and sRMSE values. HNAMs, despite their simpler architecture with shorter training times, show competitive performance and even surpass TFT in the Walmart dataset.

Table 3
Mean and median error metrics in Favorita. Superscripts identify groups via a Nemenyi test on RMSE.

	RMSSE		sRMSE		Rank freq.	
	$\bar{x}$	$\tilde{x}$	$\bar{x}$	$\tilde{x}$	1st	2nd
HNAM[1]	1.140	0.470	0.846	0.407	21.2%	**30.4%**
TFT[1]	**1.119**	**0.457**	**0.829**	**0.397**	**43.6%**	23.3%
ETS[2]	1.262	0.496	0.897	0.433	12.3%	14.9%
ETSX[3]	1.267	0.501	0.903	0.437	4.7%	9.8%
SARIMAX[4]	1.245	0.554	0.944	0.482	2.6%	2.7%
Prophet[2]	1.216	0.503	0.895	0.439	14.6%	17.1%
Lasso[5]	1.300	0.574	0.972	0.497	0.9%	1.7%

Table 4
Mean and median error metrics in Walmart. Superscripts identify groups via a Nemenyi test on RMSE.

	RMSSE		sRMSE		Rank freq.	
	$\bar{x}$	$\tilde{x}$	$\bar{x}$	$\tilde{x}$	1st	2nd
HNAM[1]	**1.050**	**0.625**	**0.853**	**0.505**	34.3%	**38.4%**
TFT[1]	1.064	0.653	0.860	0.528	**36.9%**	23.7%
ETS[1]	1.093	0.648	0.882	0.526	18.2%	24.0%
ETSX[2]	1.155	0.683	0.935	0.553	1.2%	2.4%
SARIMAX[3]	1.207	0.713	0.980	0.577	0.7%	1.4%
Prophet[3]	1.256	0.720	1.010	0.584	4.3%	3.8%
Lasso[2]	1.140	0.694	0.928	0.559	4.3%	6.2%

The rank frequencies support these observations, with TFT and HNAMs achieving the first and second ranks most frequently across the time series. This highlights the neural networks' adequacy for the task at hand with multiple, potentially non-linear and interdependent covariates and their ability to cross-learn such relationships across time series.

Interestingly, models like SARIMAX, ETSX, Prophet, and lasso, despite utilizing covariates, do not consistently outperform the univariate ETS model. This underscores the robustness of ETS as a benchmark, suggesting that adding covariates to linear models does not necessarily enhance forecast accuracy.

A Friedman test confirms that differences in model performance are statistically significant. Nemenyi post hoc tests reveal groups of models with significantly different performance ($\alpha = 0.05$). Groups are indicated with superscripts in Tables 3, 4, and 5. Neural networks are always distinct from the statistical benchmarks, except for ETS in Walmart. HNAMs are not statistically different from TFT in Walmart and Favorita, but worse than TFT in Retail.

These results confirm that the considerably less flexible but interpretable HNAM architecture is competitive with a state-of-the-art model which is considerably more complex and computationally costly.

6.3. Interpretability

After confirming HNAMs' competitive accuracy at the macro-level, this section examines HNAM interpretability outputs on various aggregation levels. HNAM explanations are natively at the instance level; that is, they

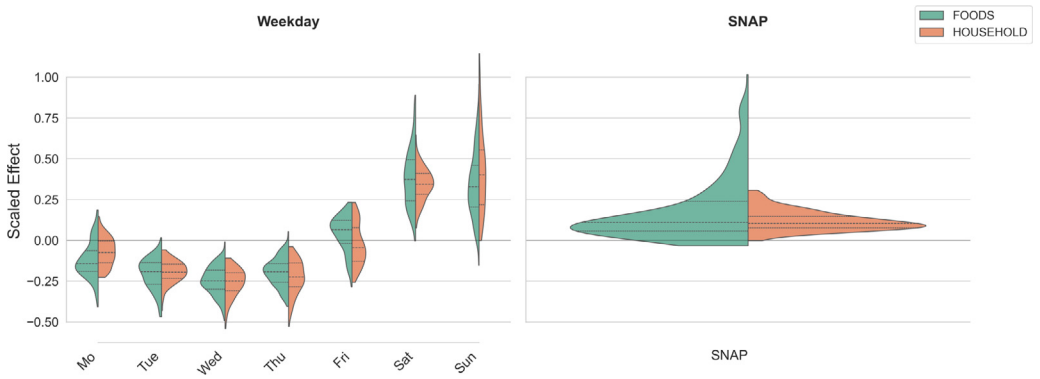


Fig. 2. Global effects in Walmart for weekdays and SNAP in two product categories.

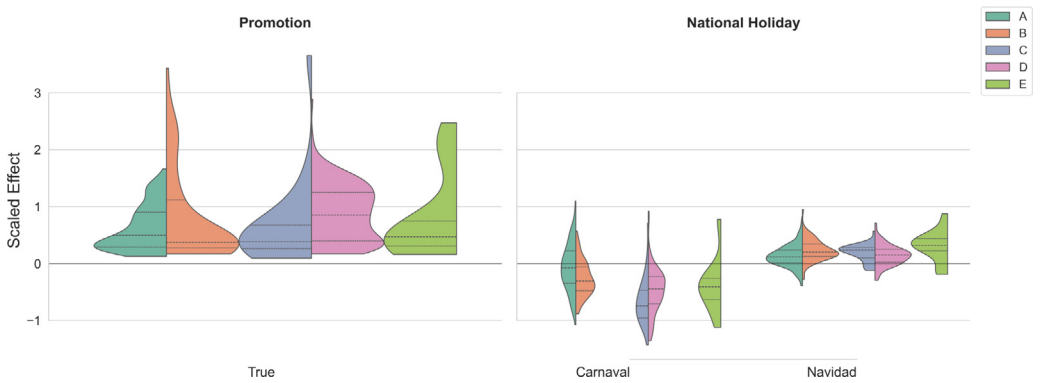


Fig. 3. Global effects in Favorita for promotions and national holidays in the five product categories.

Table 5
Mean and median error metrics in Retail. Superscripts identify groups via a Nemenyi test on RMSE.

	RMSSE		sRMSE		Rank freq.	
	$\bar{x}$	$\tilde{x}$	$\bar{x}$	$\tilde{x}$	1st	2nd
HNAM[2]	1.408	0.520	0.944	0.396	14.6%	**48.3%**
TFT[1]	**1.316**	**0.493**	**0.889**	**0.373**	**67.4%**	15.7%
ETS[3]	1.536	0.566	1.045	0.437	4.9%	5.4%
ETSX[3]	1.517	0.563	1.027	0.432	3.7%	4.6%
SARIMAX[3]	1.537	0.612	1.193	0.460	1.4%	2.9%
Prophet[3]	1.601	0.561	1.038	0.426	6.6%	19.7%
Lasso[4]	1.627	0.614	1.112	0.463	1.4%	3.4%

explain each local forecast instance with respect to the available covariates and the baseline level. Yet, we can form arbitrary aggregations to arrive at dataset-level insights (Murdoch et al., 2019).

For these, we let a trained model predict the complete dataset and aggregate the covariate-specific effects measured in standard deviations per time series.

Fig. 2 depicts the distributions of two influential causal covariates in Walmart differentiated by product category. The results confirm intuition: We observe the typical weekly sales pattern with a substantial uptake towards the weekend accompanied by increased variance across products. There is a notable asymmetry by product categories for the first days of the week, indicating relatively fewer sales in the foods category. Considering the effects of SNAP (marking the distribution of food stamps), we note a relatively higher uptake for the subsidized foods category, but customers also tend to buy more household items along the way. Similarly, Fig. 3 shows the heterogeneous effect distributions of promotions and national holidays in the five product categories in Favorita.

When sanity-checking the effects of active promotions, we observe a positive effect in 100% of promotions. The average increase relative to the level is 58% in Favorita and 154% in Retail. The provision of food stamps in Walmart via SNAP does not necessarily increase sales for all products, yielding 93% positive estimated effects. Price effects turn out to be insignificant in all datasets, indicating that the binary promotion flags capture pricing effects sufficiently. Price effects rarely exceed a few units, suggesting that pricing appears endogenously driven and does not yield predictive value beyond the promotion flags in the tested datasets.

Next, in Fig. 4, we zoom in on the global dynamics of one time series, selected for its high frequency and the effects of promotional activity across the yearly cycle in Walmart. To that end, we average the mean effects and level estimates observed on the days of a year for a forecast horizon of one week.

Here, we find the typical pattern of weekday effects, which is relatively stable throughout the year with phases of lower and higher variance in the middle of the year. Further, we find that promotional effects are highly dependent on the weekday but also change throughout the

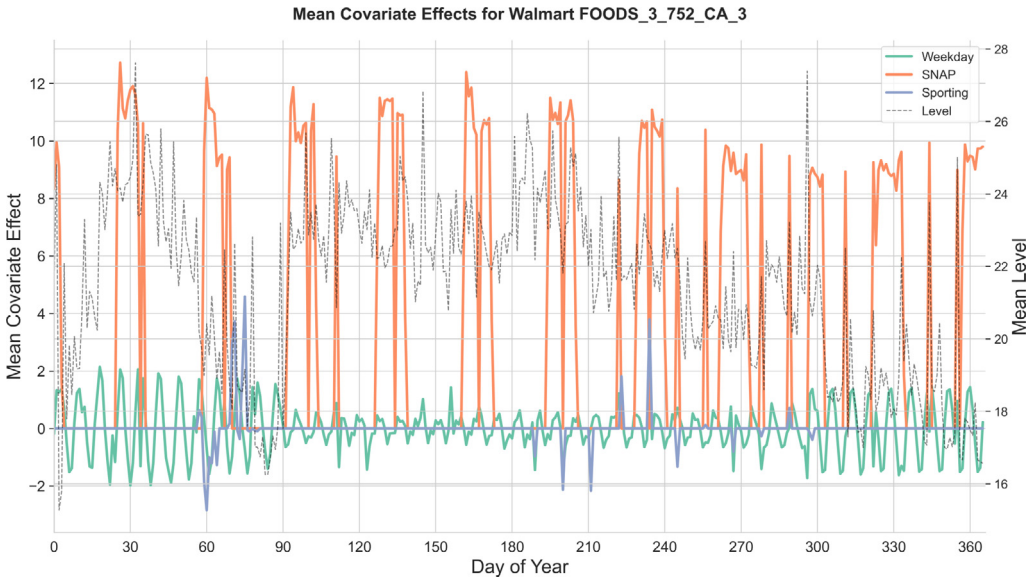


Fig. 4. Mean effects of weekdays, SNAP, and sporting events, as well as the level, for one time series in Walmart per day of the year.

year. National holidays naturally occur less often and have mixed effects on sales. Again, these results confirm expectations and demonstrate the HNAM's unique combination of interpretable yet non-linear and interaction-sensitive modeling.

Finally, we examine specific forecasting instances to demonstrate HNAMs' native explanation level and to contrast their predictive accuracy with that of TFT. All instances depict a forecast horizon of two weeks. The first instance depicted in Fig. 5 is selected for having the most covariate effects apart from the weekday effect. The forecast horizon entails Christmas, New Year, and promotions. Both methods overshoot around Christmas yet accurately capture the initial decline and subsequent uptake towards New Year. Note that the HNAM's promotion estimates vary per weekday, and that holiday effects are contingent on weekdays and promotions. The level component is flat, indicating that the respective component did not identify any trend from the sales history and only applies single exponential smoothing.

Fig. 6 visualizes an instance where HNAMs perform best relative to TFT. Interestingly, comparing the two predictions, their general shape is similar. However, TFT wrongly picks up an upward trend that HNAMs do not. The HNAM level estimate correctly predicts no change and adds the effects of weekday as well as St. Patrick's Day and Easter, both cultural covariates.

In Fig. 7, we present an instance where HNAMs fare worst relative to TFT. Like the previous example, the neural networks largely agree on their forecast shapes. In this case, however, HNAMs' conservative level estimate fails to forecast the actual decline, which is well captured by TFT.

From the discussed examples and others we screened, a wrong forecast of the general sales level, not the covariate effects, appears to be the primary driver of forecast error in both models.

6.4. Hierarchy robustness

While we recommend building the hierarchy by ordering covariates in descending frequency of their activity, practical considerations may require a custom ordering. To check its effect, we re-ran the Walmart experiment with a reversed hierarchy. We expected accuracy results not to differ significantly, since the model's expressivity and functioning are not fundamentally changed. However, explanations, which are effect estimates, are expected to vary, due to their reversed conditioning. For example, weekday effects have to account for the weekday-specific modulations of all other covariates, since these are not conditioned on the weekday anymore.

Quantitative measures confirm these views: The sRMSE metrics are mostly identical, with 0.848 for the reversed hierarchy versus 0.846. Even the training process reveals very similar loss scores. The covariate effects vary between hierarchies. However, compared with the proposed hierarchy's internal standard deviation of active effects, the deviations are notably smaller. To quantify this relation, Table 6 compares the deviation in active effect sizes between the hierarchies in the first column, reported as root mean squared deviations (RMSDs). Further, it provides the standard deviation (SD) of covariate effects within the proposed hierarchy in the second column for scale. The effect sizes vary considerably more within a hierarchy for different time steps and products than they do between two hierarchies. As mentioned above, different hierarchies do produce different effect sizes, since the meaning of covariate effects is changed. The stable accuracy results, however, indicate that rearranging the hierarchy to obtain different interpretations is not detrimental to predictive accuracy.

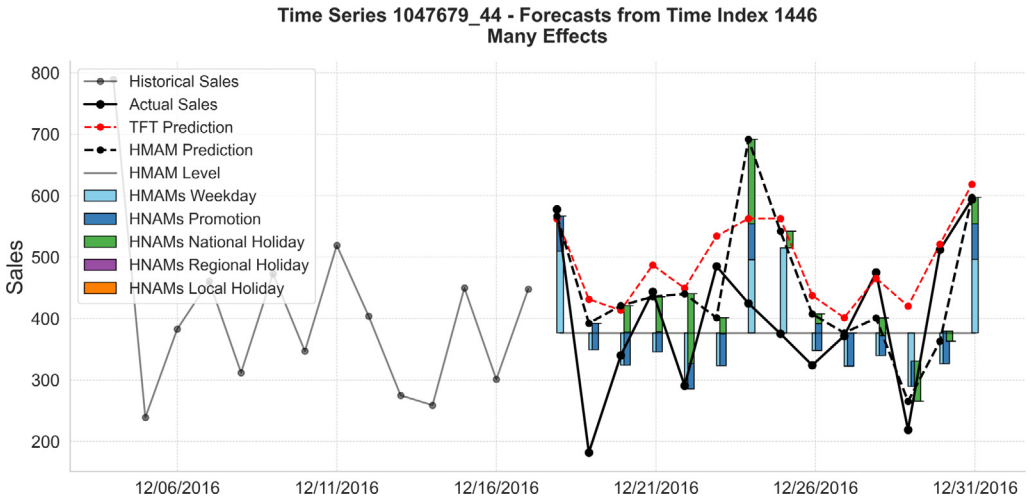


Fig. 5. Composed HNAM predictions and TFT predictions versus actuals in Favorita with many active covariates.

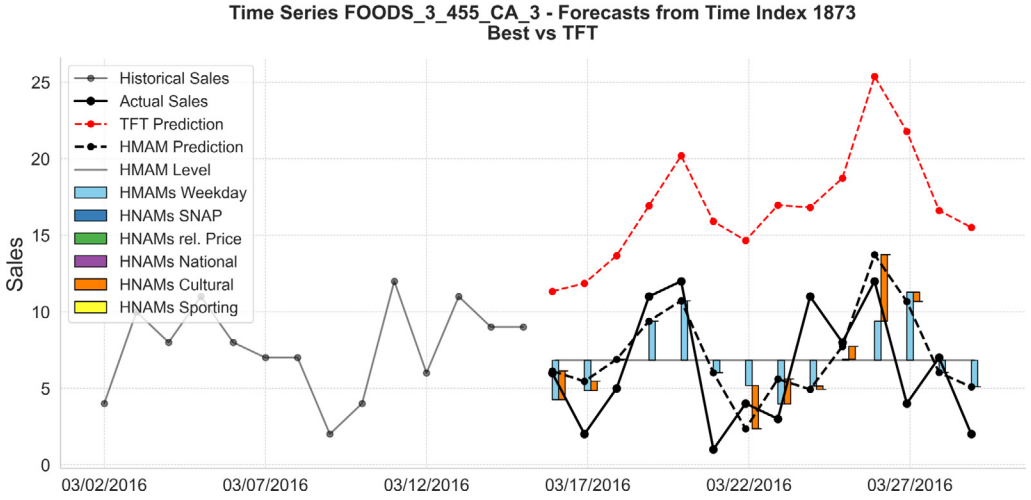


Fig. 6. Composed HNAM predictions and TFT predictions versus actuals in Walmart, where HNAMs perform the best against TFT.

Table 6
Effect sizes: Mean deviation and standard deviation.

Effect	RMSD between hierarchies	SD within proposed hierarchy
Level	0.079	0.470
Weekday	0.096	0.578
Snap	0.122	0.886
National	0.190	0.733
Cultural	0.188	0.622
Sporting	0.273	0.423
Rel. price	0.128	0.376

6.5. Horizon robustness

As a multi-horizon model, HNAMs forecast the same target date from different forecast origins. If estimated effects change drastically while approaching the target date, the model may seem less robust and trustworthy.

Therefore, we test the stability of the effects across the horizon. To that end, we set the one-day forecast (i.e., a horizon of one) as a baseline and calculate the deviations with longer horizons as percentage change relative to the sum of absolute effects for the final forecast. Thereby, the resulting percentage scores reflect the variational influence of an effect on the final forecast across the horizon.

In Fig. 8, the ratios in the title of each effect indicate its average share in the total effects for one-day forecast instances. In Panel (a), we observe that the weekday, on average, contributes 11% to the final forecast in Walmart. The distribution of percentage point differences across the horizon is graphically shown and hints at no major variation. The largest average contribution comes from the level component, leading to the highest variation with

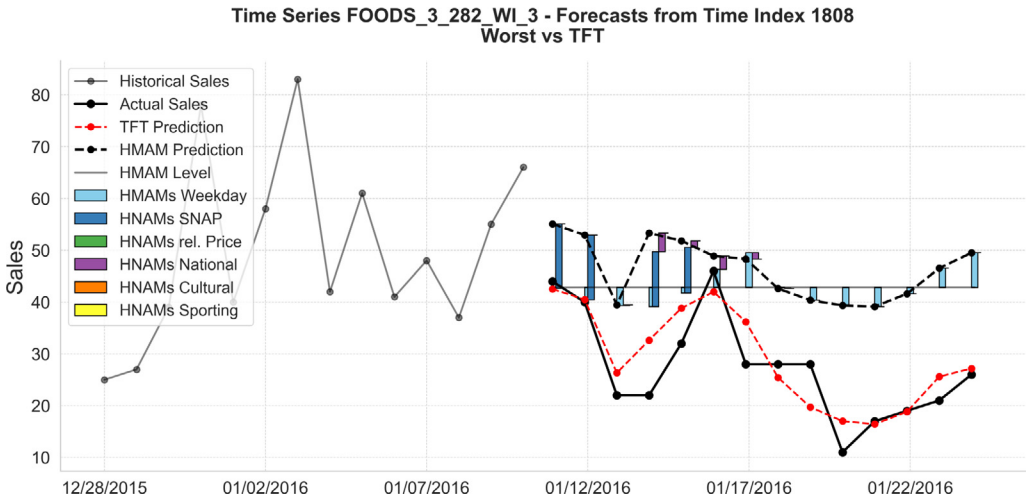


Fig. 7. Composed HNAM predictions and TFT predictions versus actuals in Walmart, where HNAMs perform the worst against TFT.

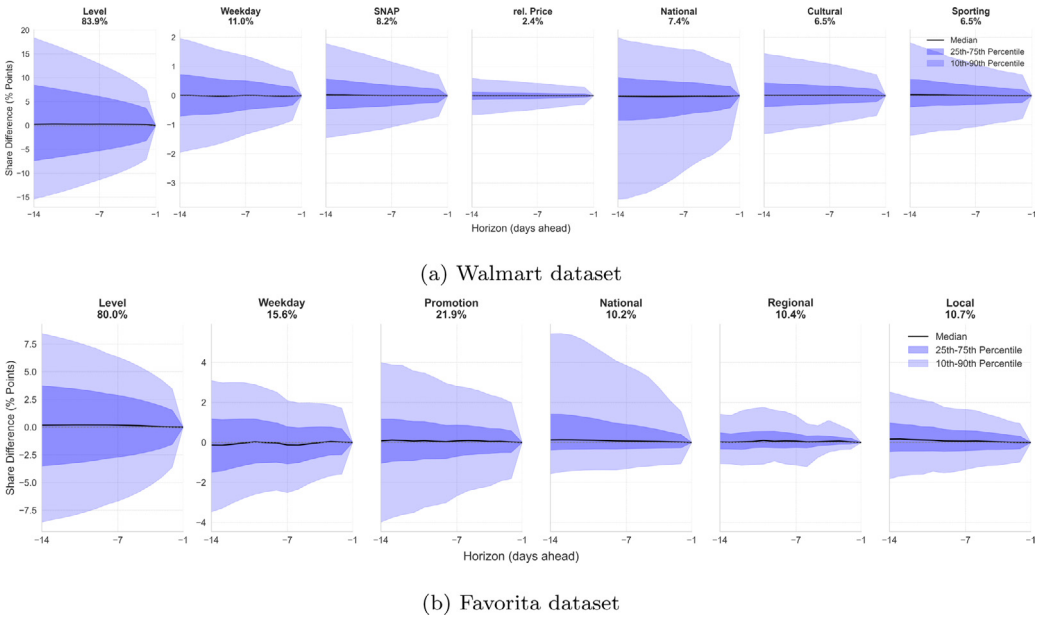


Fig. 8. Relative contributions of each covariate effect and level to the one-day-ahead forecast and distribution thereof for all horizons. Titles indicate the average relative contribution per effect for one-day-ahead forecasts. Distributional plots show variation thereof across the forecast horizon.

an increasing horizon. An analysis of forecast instances shows that sudden, erratic shifts in the time series have a relatively strong impact on level forecasts, which accounts for the greater variability observed in the extreme quantiles.

As an additional robustness check, we test whether the effect signs are constant across the horizon. This is true for more than 99% of instances per covariate and dataset, except for 98.6% for weekday in Favorita.

7. Conclusion

This study introduced hierarchical neural additive models (HNAMs) for time series forecasting, emphasizing the critical aspect of interpretability in demand forecasting models. The computational study demonstrated that HNAMs deliver competitive forecasting accuracy when benchmarked against advanced models such as temporal fusion transformers across multiple datasets. This is particularly noteworthy, as HNAMs incorporate design elements to enhance interpretability that could potentially restrict the models' expressive power. Additionally, HNAMs' composed forecasts allow users to attribute precisely how different covariates impact the forecast outcomes. This interpretability enables user interfaces that foster trust and encourage the productive injection of extra-model knowledge.

7.1. Limitations and challenges in practice

While HNAMs represent a significant step towards inherently interpretable models for demand forecasting, there are potential challenges associated with their practical implementation for decision support:

Covariate causality misinterpretation. Although the HNAM architecture clearly maps the information used to estimate covariate effects, it does not establish causality. For instance, a dataset containing promotions and price changes can produce counterintuitive effects if not correctly interpreted: A product that only has price changes concurrent with promotions might show a negative effect of the price change if it is less extensive than other price changes during promotions. Similarly, past price increases as reactions to increasing demand could imply higher demand for higher prices. In the results of the computational study documented in this paper, price effects were, however, of very low magnitude compared to promotion effects. The outlined caveats are not unique to HNAMs but apply to any statistical reasoning. However, they emphasize the importance of training when deploying interpretable models in practice.

Architectural limitations. The proposed HNAM implementation, while utilizing best practices such as layer normalization, dropout, and multi-head attention, is not the result of an exhaustive architecture search. There may be more efficient and more accurate ways to implement the general idea of HNAMs that have yet to be explored.

Delayed and anticipatory effects. HNAMs, as proposed in this paper, do not natively learn to model delayed or anticipatory effects, such as when a promotion affects subsequent periods (Hewage et al., 2022) or when customers anticipate events or holidays. We have included the five days leading up to an event or holiday as features, but optimally, such feature engineering would be automated. Future work is needed to develop neural architectural solutions that learn such time-shifted effects while maintaining interpretability.

7.2. Future research directions

Behavioral validation. The potential of HNAMs to improve HCI in retail demand forecasting rests on arguments laid out in Section 2. However, no study to date has empirically examined analysts' interactions with inherently explainable models like HNAMs, compared to black-box models complemented by post hoc explanations. Future research should include behavioral studies with domain experts and larger-scale evaluations with business students to assess how explainable models affect decision-making processes and outcomes in multivariate forecasting tasks.

Architectural improvements. Further research is encouraged to explore different architectural implementations of HNAMs. Investigating more efficient structures or components could enhance both their accuracy and interpretability. Furthermore, future iterations should inherently model delayed and anticipatory effects while maintaining clear attributions.

7.3. Final remarks

As organizations navigate data-rich environments and confront challenges related to human–computer interaction, our findings demonstrate that HNAMs represent a significant advancement toward more effective decision making through accurate and inherently interpretable models. We encourage future research to refine this architecture and explore its interplay with decision makers. This work, therefore, holds great potential to drive the successful integration of advanced forecasting models into practical applications.

CRediT authorship contribution statement

Leif Feddersen: Writing – original draft, Visualization, Validation, Software, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Catherine Cleophas:** Writing – review & editing, Supervision, Resources, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix. Hyperparameter optimization

To fine-tune the performance of both the HNAMs and TFT models, we conducted a hyperparameter optimization using the last two months of the training data as a validation set before running the main experiment. We performed a grid search over a predefined set of hyperparameters specific to each model's architecture. Table A.7 summarizes the hyperparameters and their respective search spaces.

The shared hyperparameters between HNAMs and TFTs included the learning rate, batch size, maximum gradient norm, encoder length (in weeks), dropout rate, and embedding size. HNAMs further had the expansion factor as a hyperparameter.

We performed a random search over the hyperparameter grid and recorded validation scores on the holdout sample for 24 h on an H100 high-performance computing node. Models trained for a maximum of 100 epochs with early stopping, using a patience of 20 epochs, to predict the first validation month. For the second validation month, the first validation month was added to the training data for fine-tuning, and models were trained for up to 30 additional epochs.

The best-scoring model across the two validation months was used for the experiment.

Data and code sharing information

To support reproducibility, we provide the following GitHub repository containing all code used for our analysis: https://github.com/leif-feddersen/HNAMs-Demand-Forecasting.

Table A.7
Hyperparameter search space for HNAMs and TFT models.

Hyperparameter	Values
Learning rate	[1e−2, 1e−3]
Batch size	[128, 256]
Max gradient norm	[0.1, 1, 100]
Encoder length (weeks)	[4, 12]
Dropout rate	[0.1, 0.3, 0.5]
Attention heads	[1, 4]
TFTs: Hidden size	[32, 128, 240]
HNAMs: Embedding size	[8, 32, 64]
HNAMs: Expansion factor	[1, 2, 4]

References

Agarwal, R., Melnick, L., Frosst, N., Zhang, X., Lengerich, B., Caruana, R., & Hinton, G. E. (2021). Neural additive models: Interpretable machine learning with neural nets. *Advances in Neural Information Processing Systems*, *34*, 4699–4711.

Alroomi, A., Karamatzanis, G., Nikolopoulos, K., Tilba, A., & Xiao, S. (2022). Fathoming empirical forecasting competitions' winners. *International Journal of Forecasting*, *38*, 1519–1525.

Antipov, E. A., & Pokryshevskaya, E. B. (2020). Interpretable machine learning for demand modeling with high-dimensional data using gradient boosting machines and Shapley values. *Journal of Revenue and Pricing Management*, *19*, 355–364.

Beitner, J. (2020). GitHub - jdb78/pytorch-forecasting: Time series forecasting with PyTorch. https://github.com/jdb78/pytorch-forecasting. (Accessed on 3 April 2024).

Box, G. E., Jenkins, G. M., Reinsel, G. C., & Ljung, G. M. (2015). *Time series analysis: forecasting and control*. John Wiley & Sons.

Burton, J. W., Stein, M. K., & Jensen, T. B. (2020). A systematic review of algorithm aversion in augmented decision making. *Journal of Behavioral Decision Making*, *33*, 220–239.

Carbone, R., & Gorr, W. L. (1985). Accuracy of judgmental forecasting of time series. *Decision Sciences*, *16*, 153–160.

Chatfield, C. (1993). Neural networks: forecasting breakthrough or passing fad? *International Journal of Forecasting*, *9*, 1–3.

Cho, K. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. arXiv preprint arXiv: 1406.1078.

Corporación Favorita (2018). *Corporación favorita grocery sales forecasting competition*, https://www.kaggle.com/c/favorita-grocery-sales-forecasting/.

Dietvorst, B. J., Simmons, J. P., & Massey, C. (2015). Algorithm aversion: People erroneously avoid algorithms after seeing them err. *Journal of Experimental Psychology: General*, *144*(114).

Dietvorst, B. J., Simmons, J. P., & Massey, C. (2018). Overcoming algorithm aversion: People will use imperfect algorithms if they can (even slightly) modify them. *Management Science*, *64*, 1155–1170.

Edmundson, B., Lawrence, M., & O'Connor, M. (1988). The use of non-time series information in sales forecasting: A case study. *Journal of Forecasting*, *7*, 201–211.

Feddersen, L. (2024). Algorithmic transparency in forecasting support systems. URL https://arxiv.org/abs/2411.00699, arXiv:2411.00699.

Feddersen, L., & Cleophas, C. (2023). Solving the data-driven newsvendor with attention to time. In *ECIS 2023 proceedings*.

Fildes, R., & Goodwin, P. (2007). Against your better judgment? How organizations can improve their use of management judgment in forecasting. *Interfaces*, *37*, 570–576.

Fildes, R., & Goodwin, P. (2021). Stability in the inefficient use of forecasting systems: A case study in a supply chain company. *International Journal of Forecasting*, *37*, 1031–1046.

Fildes, R., Goodwin, P., Lawrence, M., & Nikolopoulos, K. (2009). Effective forecasting and judgmental adjustments: An empirical evaluation and strategies for improvement in supply-chain planning. *International Journal of Forecasting*, *25*, 3–23.

Fildes, R., Kolassa, S., & Ma, S. (2022a). Post-script–retail forecasting: Research and practice. *International Journal of Forecasting*, *38*, 1319–1324.

Fildes, R., Ma, S., & Kolassa, S. (2022b). Retail forecasting: Research and practice. *International Journal of Forecasting*, *38*, 1283–1318.

Fildes, R., Schaer, O., Svetunkov, I., & Yusupova, A. (2020). Survey: What's new in forecasting software? *OR/MS Today*, *47*.

Garza, F., Mergenthale. Canseco, M., Challú, C., & Olivares, K. G. (2022). Statsforecast: Lightning fast forecasting with statistical and econometric models. https://github.com/Nixtla/statsforecast. (Accessed on 3 April 2024).

Gilpin, L. H., Bau, D., Yuan, B. Z., Bajwa, A., Specter, M., & Kagal, L. (2018). Explaining explanations: An overview of interpretability of machine learning. In *2018 IEEE 5th international conference on data science and advanced analytics* DSAA, (pp. 80–89). IEEE.

Goodwin, P. (2002). Integrating management judgment and statistical methods to improve short-term forecasts. *Omega*, *30*, 127–135.

Goodwin, P., & Fildes, R. (2022). Forecasting with judgment. In *The Palgrave handbook of operations research* (pp. 541–572). Springer.

Hewage, H. C., Perera, H. N., & D. Baets, S. (2022). Forecast adjustments during post-promotional periods. *European Journal of Operational Research*, *300*, 461–472.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, *9*, 1735–1780.

Jo, W., & Kim, D. (2023). Neural additive time-series models: Explainable deep learning for multivariate time-series prediction. *Expert Systems with Applications*, *228*, Article 120307.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T. Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, *30*.

Khosrowabadi, N., Hoberg, K., & Imdahl, C. (2022). Evaluating human behaviour in response to AI recommendations for judgemental forecasting. *European Journal of Operational Research*, *303*, 1151–1167.

Klassen, R. D., & Flores, B. E. (2001). Forecasting practices of Canadian firms: Survey results and comparisons. *International Journal of Production Economics*, *70*, 163–174.

Lawrence, M. J., Edmundson, R. H., & O'Connor, M. J. (1985). An examination of the accuracy of judgmental extrapolation of time series. *International Journal of Forecasting*, *1*, 25–35.

Lehmann, C. A., Haubitz, C. B., Fügener, A., & Thonemann, U. W. (2022). The risk of algorithm transparency: How algorithm complexity drives the effects on the use of advice. *Production and Operations Management*, *31*, 3419–3434.

Lim, B., Arık, S. Ö., Loeff, N., & Pfister, T. (2021). Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, *37*, 1748–1764.

Lipton, Z. C. (2018). The mythos of model interpretability: In machine learning, the concept of interpretability is both important and slippery. *Queue*, *16*, 31–57.

Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, *30*.

Lütkepohl, H. (2005). *New introduction to multiple time series analysis*. Springer.

Makridakis, S., Andersen, A., Carbone, R., Fildes, R., Hibon, M., Lewandowski, R., Newton, J., Parzen, E., & Winkler, R. (1982). The accuracy of extrapolation (time series) methods: Results of a forecasting competition. *Journal of Forecasting*, *1*, 111–153.

Makridakis, S., & Hibon, M. (2000). The M3-competition: results, conclusions and implications. *International Journal of Forecasting*, *16*, 451–476.

Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2018a). Statistical and machine learning forecasting methods: Concerns and ways forward. *PLoS One*, *13*, Article e0194889.

Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2018b). The M4 competition: results, findings, conclusion and way forward. *International Journal of Forecasting*, *34*, 802–808.

Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2022). M5 accuracy competition: Results, findings, and conclusions. *International Journal of Forecasting*, *38*, 1346–1364.

Marmier, F., & Cheikhrouhou, N. (2010). Structuring and integrating human knowledge in demand forecasting: A judgemental adjustment approach. *Production Planning and Control*, *21*, 399–412.

Mello, J. (2009). The impact of sales forecast game playing on supply chains. *Foresight: The International Journal of Applied Forecasting*, *13*, 13–22.

Mentzer, J. T., & Moon, M. A. (2004). *Sales forecasting management: a demand management approach*. Sage Publications.

Montero-Manso, P., & Hyndman, R. J. (2021). Principles and algorithms for forecasting groups of time series: Locality and globality. *International Journal of Forecasting, 37*, 1632–1653.

Murdoch, W. J., Singh, C., Kumbier, K., Abbasi-Asl, R., & Yu, B. (2019). Definitions, methods, and applications in interpretable machine learning. *Proceedings of the National Academy of Sciences*, *116*, 22071–22080.

Neslin, S. A., & Shankar, V. (2009). Key issues in multichannel customer management: Current knowledge and future directions. *Journal of Interactive Marketing*, *23*, 70–81.

Petropoulos, F., Fildes, R., & Goodwin, P. (2016). Do 'big losses' in judgmental adjustments to statistical forecasts affect experts' behaviour? *European Journal of Operational Research*, *249*, 842–852.

Prahl, A., & Van Swol, L. (2017). Understanding algorithm aversion: When is advice from automation discounted? *Journal of Forecasting*, *36*, 691–702.

Qi, M., Shi, Y., Qi, Y., Ma, C., Yuan, R., Wu, D., & Shen, Z. J. (2023). A practical end-to-end inventory management model with deep learning. *Management Science*, *69*, 759–773.

Ramos, P., Oliveira, J., Fildes, R., & Ma, S. (2017). Characterizing retail demand with promotional effects for model selection. In *37th international symposium on forecasting*.

Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why should I trust you?": Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 1135–1144).

Rudin, C. (2019). Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, *1*, 206–215.

Salinas, D., Flunkert, V., Gasthaus, J., & Januschowski, T. (2020). DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting, 36*, 1181–1191.

Sanders, N. R., & Manrodt, K. B. (2003). The efficacy of using judgmental versus quantitative forecasting methods in practice. *Omega*, *31*, 511–522.

Sanders, N. R., & Ritzman, L. P. (1992). The need for contextual and technical knowledge in judgmental forecasting. *Journal of Behavioral Decision Making*, *5*, 39–52.

Schaer, O., Svetunkov, I., Yusupova, A., & Fildes, R. (2022). Survey: Forecasting software trends in a challenging world. *OR/MS Today*.

Smyl, S. (2020). A hybrid method of exponential smoothing and recurrent neural networks for time series forecasting. *International Journal of Forecasting, 36*, 75–85.

Taylor, S. J., & Letham, B. (2017). Github - facebook/prophet: Prophet: Automatic forecasting procedure. https://github.com/facebook/prophet. (Accessed on 3 April 2024).

Taylor, S. J., & Letham, B. (2018). Forecasting at scale. *The American Statistician*, *72*, 37–45.

Trapero, J. R., Pedregal, D. J., Fildes, R., & Kourentzes, N. (2013). Analysis of judgmental adjustments in the presence of promotions. *International Journal of Forecasting, 29*, 234–243.

Ulrich, M., Jahnke, H., Langrock, R., Pesch, R., & Senge, R. (2021). Distributional regression for demand forecasting in E-grocery. *European Journal of Operational Research*, *294*, 831–842.

University of Nicosia (2020). *M5 forecasting - accuracy: Estimate the unit sales of walmart retail goods*, https://www.kaggle.com/c/m5-forecasting-accuracy.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, *30*.

Vössing, M., Kühl, N., Lind, M., Satzger, G., & AI Collaboration (2022). Designing transparency for effective human-AI collaboration. *Information Systems Frontiers*, *24*, 877–895.